

En este documento se hará un estudio sobre el último informe de la OWASP acerca de las 10 vulnerabilidades más comunes en las páginas webs, que data del 2025.

A01 Pérdida de control de acceso	1
A02 Fallos en la configuración de seguridad	2
A03 Fallo en la cadena de suministro de software.....	3
A04 Fallos criptográficos	4
A05 Inyección	5
A06 Diseño inseguro.....	6
A07 Fallos de autenticación	7
A08 Fallos en la integridad de los datos o de software.....	9
A09 Fallos en el registro y en el monitoreo.....	10
A10 Fallos en el manejo de condiciones excepcionales	11

A01 Pérdida de control de acceso

Manteniendo su primer puesto como en años anteriores, la pérdida del control de acceso es la vulnerabilidad más común. Según el estudio de la OWASP, el 100% de las páginas web tienen algún fallo relacionado con este aspecto.

El control de acceso obliga ciertas conductas para que los usuarios no puedan actuar más allá de sus permisos. alguna de las vulnerabilidades más comunes:

- Violación del principio del último privilegio, donde el acceso solo debería ser dado para capacidades, roles o usuarios particulares; sin embargo, es accesible para todos.
- Bypassing el control de acceso mediante la modificación de la URL, el estado interno de la aplicación, de la página HTML o con una herramienta que modifica las peticiones API
- Permitiendo la visualización o edición la cuenta de un tercero por dar su identificador único (referencias a objetos inseguras)
- Una API accesible sin controles de acceso para POST, PUT y DELETE

Recomendaciones de cómo prevenir algunas de las vulnerabilidades.

- Salvo por los recursos públicos, deniega por defecto
- Implementar mecanismos de control de acceso una vez u rehusarlos en el resto de la aplicación, minimizando el uso del [CORS](#)
- El control de acceso debe implementar su cumplimiento a nivel de dato y no permitir que el usuario pueda crear, leer, actualizar o borrar cualquier dato.
- Registrar los fallos de control de acceso cuando sean necesarios (muchas veces el mismo fallo)

Para ver más peligros, más formas de prevenirlos y ejemplos de escenarios de ataque sobre el control de acceso:

https://owasp.org/Top10/2025/A01_2025-Broken_Access_Control/

A02 Fallos en la configuración de seguridad

Esta vulnerabilidad comparte con la primera que todas las webs testeadas por la OWASP han tenido alguno fallo de configuración.

Alguna de las vulnerabilidades posibles por una mal configuración:

- Hay características innecesarias instaladas o incluso habilitadas (puertos, servicios, cuentas, privilegios, etc)
- Las cuentas por defecto y sus contraseñas están habilitadas y no modificadas.
- Falta de “[security hardening](#)” en alguna parte de la aplicación o permisos mal configurados en los servicios de la nube
- Para sistemas actualizados, las ultimas características de seguridad están desactivadas o no configuradas de forma segura

Algunas medidas de prevención:

- Entornos de Desarrollo, [QA](#) y producción deben de estar configurados de forma idéntica, pero con distintas credenciales
- Una plataforma minimalista sin componentes ni características innecesarias. (Desinstalar todo aquello que sea innecesario)
- Enviar directivas de seguridad a los clientes

https://owasp.org/Top10/2025/A02_2025-Security_Misconfiguration/

A03 Fallo en la cadena de suministro de software

Son fallos en la cadena de suministro de software u otros compromisos durante el proceso de construcción. Normalmente, se producen por vulnerabilidades en código third-party, herramientas u otras dependencias de los que el sistema depende.

Alguna de las vulnerabilidades:

- No tienes un seguimiento de las versiones de tus componentes.
- El software es vulnerable, sin soporte o desactualizado.
- No escaneas para buscar vulnerabilidades regularmente

Algunas medidas de prevención:

- No solo hagas seguimiento de tus dependencias directas, sino de las dependencias de tus dependencias
- Elimina todo lo que no uses (dependencias, archivos, documentacion...)
- Consulta regularmente las versiones de los componentes tanto del lado del servidor como del lado del cliente, y sus dependencias.
- Solo descargar componentes de links fiables y de fuentes confiables.

Para ver más peligros, más formas de prevenirlos y ejemplos:

https://owasp.org/Top10/2025/A03_2025-Software_Supply_Chain_Failures/

A04 Fallos criptográficos

Es necesario determinar qué información necesita ser encriptada, y cuál necesita ser extra encriptada. Ejemplos evidentes son las contraseñas o las tarjetas de crédito. Ejemplos menos evidentes son los datos médicos de uno.

Algunos conceptos a los que prestar atención:

- Buscar si hay algoritmos criptográficos viejos o débiles que son usados por defecto o en código viejo
- Hay claves por defecto en uso, se generan claves criptográficas débiles o claves reutilizadas.
- La encriptación no es forzada o faltan cabeceras.
- El certificado del servidor recibido y la cadena de confianza son debidamente validadas

Algunas medidas de prevención:

- Clasifica y etiqueta los datos procesados, guardados o transmitidos por la aplicación.
- Guarda las claves más importantes en un [HSM](#)
- Siempre que sea posible, implementa algoritmos criptográficos confiables.

Para ver más peligros, más formas de prevenirlos y ejemplos:

https://owasp.org/Top10/2025/A04_2025-Cryptographic_Failures/

A05 Inyección

La inyección es un fallo en la aplicación que permite que los inputs de un usuario no seguro sean enviados a un intérprete y que este ejecute parte de esos inputs como comandos

Una aplicación es vulnerable a un ataque cuando :

- Los datos suministrados por el usuario no son validados, filtrados o saneados por la aplicación
- consultas dinámicas o llamadas sin parámetros son usadas directamente por el intérprete
- Datos no saneados son usados en el la búsqueda de parámetros del mapeo de objetos relacionales ([ORM](#)) para extraer registros adicionales y sensibles.
- Datos potencialmente hostiles son usados directamente o concatenados. El SQL o el comando contiene la estructura y los datos maliciosos en consultas dinámicas, comandos o procedimientos.

Como prevenirlos:

- La opción preferida es usar una API segura que evita el uso de intérpretes completamente, suministra una interfaz parametrizada o migra para ORMS.
- Hacer validación en el lado del servidor
- Si quedan consultas dinámicas, neutralizar los caracteres especiales usados en la sintaxis de los interpretes

Para ver ejemplos de ataques: https://owasp.org/Top10/2025/A05_2025-Injection/

A06 Diseño inseguro

En esta categoría entran todas las vulnerabilidades que en las que “falta o el control de diseño es ineficiente”.

Formas genéricas de prevenirlos:

- Establecer y usar un ciclo de vida de desarrollo seguro con profesionales que te ayuden a evaluar y diseñar los controles de seguridad y de privacidad.
- Establecer y usar una librería de patrones de diseño seguros.
- Usar un modelado de amenazas para las partes críticas de la aplicación como la autenticación, el control de acceso y lógica de negocio.
- Usar un modelado de amenazas como una herramienta educativa para generar una mentalidad sobre la seguridad.
- Integrar lenguaje de seguridad y controles sobre la [historia de usuarios](#)

Ejemplos: https://owasp.org/Top10/2025/A06_2025-Insecure_Design/

A07 Fallos de autenticación

<https://alvarogargon.ieslossauces.es/AGGCIBProyectoCIB/doc/OWASP07.pdf>

Los fallos de autenticación son aquellas vulnerabilidades que ocurren cuando un atacante es capaz de engañar al sistema para que un usuario inválido o incorrecto se reconocido como válido.

Algunos de los fallos comunes:

- Permitir los ataques de [relleno de credenciales](#) automatizadas. Estos tipos de ataques han evolucionado en el que incluyen variaciones de las contraseñas filtradas (ej: contra1, contra2, contra3, contra4)
- Uso de contraseñas débiles o reutilizadas o embebidas en el código.
- Falta de protección contra ataques de fuerza bruta
- Gestión insegura de sesiones
- Ausencia o mala implementación de autenticación multifactor (MFA).
- Validaciones incorrectas en certificados, tokens o flujos de login.
- Permitir la creación de usuarios con credenciales que se sabe que han sido filtradas.
- Usar encriptamiento débil o texto plano en el almacenamiento de contraseñas
- Permitir métodos de recuperación de contraseñas débiles como “preguntas de saber” (ej:¿Como se llamaba tu primer perro?)
- Expone el identificador de sesión en la URL, un campo oculto u otra ubicación no segura
- Reusar el mismo identificador de sesión después de un login correcto

Como prevenirlo:

- Implementar autenticación multifactor siempre que sea posible.
- Aplicar políticas robustas de contraseñas y almacenamiento seguro
- Limitar intentos de inicio de sesión.
- Cuando sea posible, animar el uso de gestores de contraseñas.
- No desplegar con credenciales por defecto, especialmente con los usuarios administradores
- Implementar chequeos de contraseñas débiles, por ejemplo, contra la lista de [“las 10.000 peores contraseñas”](#)
- Durante la creación de nuevas cuentas, validarlas contra una lista de credenciales filtradas
- No fuerces a las personas a cambiar la contraseña sino sospechas de que haya habido una filtración. En cuyo caso, oblígales.

- No dar información en el login de porque es incorrecto
- Usar un gestor de sesiones en el lado del servidor que genere una nueva ID de sesión aleatoria después del login.

https://owasp.org/Top10/2025/A07_2025-Authentication_Failures/

A08 Fallos en la integridad de los datos o de software

Son fallos en la programación y la infraestructura que no protegen contra código o datos inválidos ya que son tratados como válidos y confiables. Un caso típico de esta vulnerabilidad es cuando una aplicación depende de plugins/librerías/módulos que provienen de fuentes no verificadas.

Como prevenirlo:

- Usar firmas digitales o mecanismos similares para verificar el software o los datos provienen de la fuente esperada y sin alterar
- Asegúrate que las librerías y dependencias solo están consumiendo de repositorios confiables.
- Asegúrate que hay un proceso de revisión para los cambios del código y la configuración para minimizar las probabilidades de que código o configuración sea introducido en tu software

Ejemplos de ataques: https://owasp.org/Top10/2025/A08_2025-Software_or_Data_Integrity_Failures/

A09 Fallos en el registro y en el monitoreo

Sin los logs y el monitoreo, los ataques y las brechas de seguridad no pueden ser detectados. Además de que, aunque se detectaran, sería muy difícil presentar una respuesta de forma rápida y efectiva.

Algunos fallos que se pueden presentar en la aplicación:

- Eventos verificables, como logins, logins fallidos y transacciones de alto valor, no son registrados o lo son de forma inconsistente.
- Los errores y avisos generan mensajes de registro inadecuados, no claros o directamente no se generan.
- La integridad de los registros no es protegida debidamente.
- Los registros de las aplicaciones y de las APIs no son monitoreados para actividades sospechosas.
- Los registros solo se guardan de forma local y no hay una copia de seguridad
- La aplicación no puede detectar, escalar o alertar ataques en tiempo real
- ...

Algunas maneras de prevenirlo:

- Asegurarse que todos los fallos en logins, controles de acceso y validaciones de inputs del lado del servidor sean registrados con suficiente contexto de usuario para identificar cuentas maliciosas o sospechosas.
- Asegurarse que todos los controles de acceso sean registrados, ya sea por error o por correcto acceso.
- Asegurarse que los registros son generados en un formato en el que un gestor de registros lo puede consumir fácilmente
- Asegurarse que los datos de los registros son codificados correctamente para prevenir inyecciones o ataques en los sistemas de registro o monitoreo
- Asegurarse que todas las transacciones tienen un rastro con controles de integridad para prevenir la manipulación o eliminación
- ...

Para ver más fallos, más métodos de prevención y ejemplos de ataques:

https://owasp.org/Top10/2025/A09_2025-Security_Logging_and_Alerting_Failures/

A10 Fallos en el manejo de condiciones excepcionales

Estos fallos pasan cuando los programas fallan al detectar, prevenir y responder a situaciones inusuales o impredecibles, lo que causan crasheos, comportamientos inesperados y, algunas veces, vulnerabilidades. Los 3 fallos que ocurren en estas situaciones son:

1. La aplicación no impide que situaciones inesperadas ocurran
2. La aplicación no identifica la situación mientras que ocurre
3. No responde o responde de manera pobre a la situación

Las condiciones excepcionales pueden ser causadas por:

- Validaciones de input incompletas, pobres o inexistentes
- Manejo de errores de alto nivel en vez de en las funciones donde ocurren
- Manejo de excepciones inconsistentes

Como prevenirlo:

Para poder manejar situaciones excepcionales debemos prepararnos para dichas situaciones, lo que es lo mismo, prepararnos para lo peor. Debemos hacer 'catch' a todos los posibles errores del sistema directamente donde ocurren y manejarlos. Como parte de manejo, debemos lanzar un error para informar al usuario (de una manera entendible), registrar los eventos, así como añadir una alerta si está justificado. También deberíamos tener un manejador de excepciones globales listo en caso de que se nos haya escapado algo. Idealmente también deberíamos de disponer de herramientas de monitorización, observación o funcionalidad que miren errores que se repiten o patrones que indiquen que un ataque está ocurriendo, y que pudieran emitir una respuesta, defensa o bloqueo de algún tipo.

Para obtener más información: https://owasp.org/Top10/2025/A10_2025-Mishandling_of_Exceptional_Conditions/